

AI Large Model Preparation

AI Large Model Preparation

1. Select the matching car model
2. Radar mapping
3. Color calibration
4. Configure large model related API keys
5. Start the text interaction client

1. Select the matching car model

Open a terminal and enter:

```
gedit .bashrc
```

Find this line:

```
124
125 export CAR_TYPE=ptz          #ptz / no_ptz / basic_car  Three types
126 #echo "LOCAL_IP: $(hostname -I | awk '{print $1}')"
127 echo "===== Current IP ====="
128 #ip -o -4 addr show scope global | awk '{split($4,a,"/"); printf "%-10s %s\n",
129 $2":", a[1]}'
129 ip -o -4 addr show scope global | awk '{
130     split($4,a,"/");
131     if (a[1] ~ /^192\.168\./)
132         printf "%-10s %s\n", $2":", a[1]
133 }'
134 echo "=====
135 echo "ROS_DOMAIN_ID=${ROS_DOMAIN_ID}"
136 echo "CAR_TYPE=${CAR_TYPE}"
137 echo "=====
138
139 export NVM_DIR="$HOME/.nvm"
140 [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
141 [ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads
142 nvm bash_completion
143
144 # OpenClaw Completion
145 [ -f "/home/yahboom/.openclaw/completions/openclaw.bash" ] && source "/home/
146 yahboom/.openclaw/completions/openclaw.bash"
```

Fill in your current car model in the car_type field:

ptz: with pan-tilt servo + camera

no_ptz: no servo + with fixed camera

basic_car: no servo + no camera

After modifying, click save and exit, then run the following command to apply the changes:

```
source ~/.bashrc
```

2. Radar mapping

Refer to the content in the product tutorial 【Lidar Course】 - 【9. slam_toolx Mapping】 to start the car for mapping and save the map. This map will be used for navigation in AI large model applications later.

3. Color calibration

Start the color calibration program to calibrate and save the HSV values of the four colors red, green, blue and yellow in the parameter file. In the AI large model color line-following case later, `llm_agent_node` will load this parameter file via `FOLLOW_LINE.hsv_file`. When you input natural language commands such as "start following the red line" or "start following the yellow line" through the terminal text client, Dify will convert the commands into actions such as `follow_line(red)` and `follow_line(yellow)`.

Source code path:

```
~/microros_ws/src/multi_brains/multi_brains/utils/colorSelect.py
```

Parameter save file path:

```
~/microros_ws/src/multi_brains/config/color_hsv.txt
```

Later, if the large model needs color-related data, it will read the HSV values of colors from this path.

Start the camera:

```
# Start proxy + camera
ros2 launch microros_v2_bringup car_base.launch.py
```

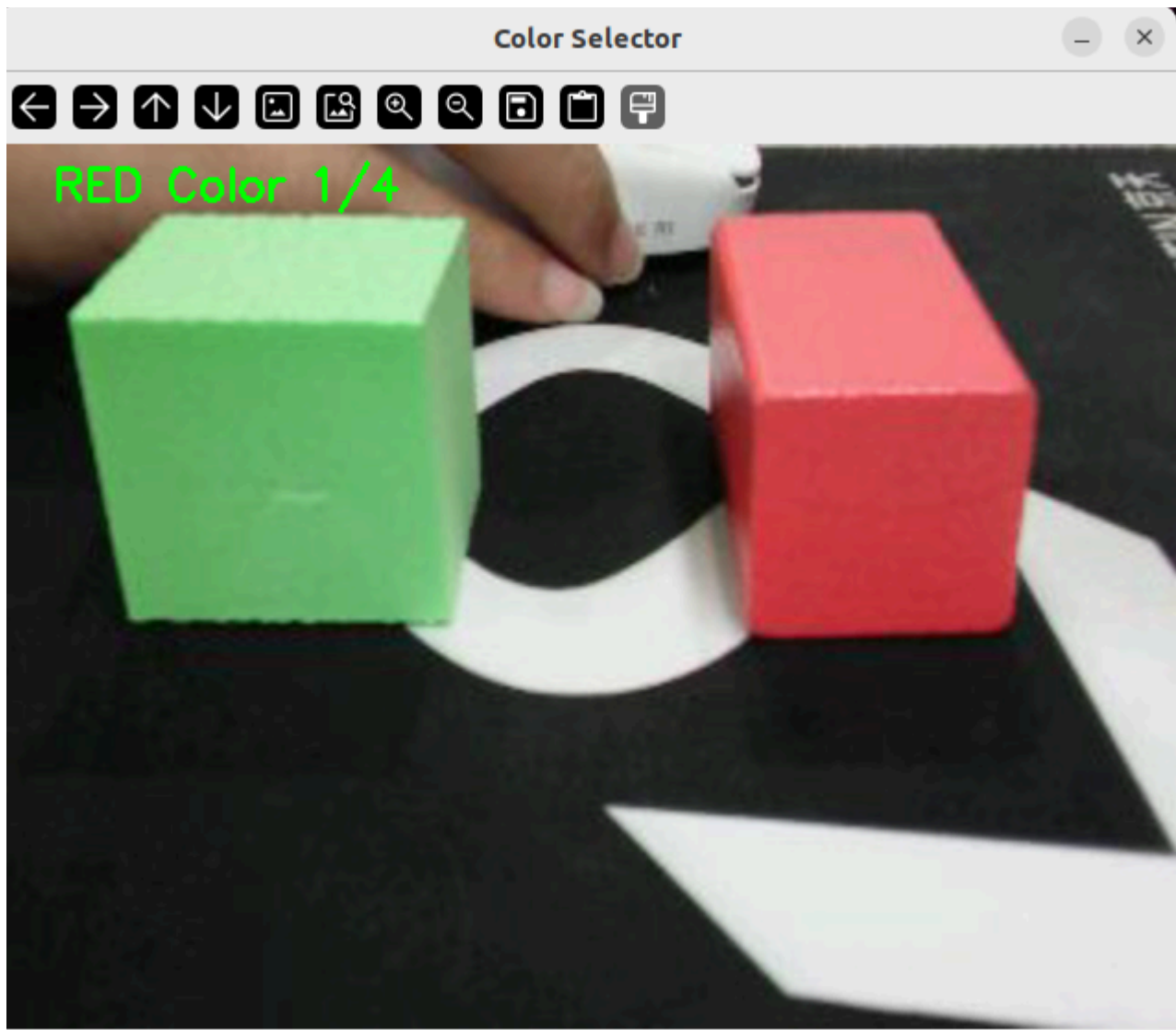
```
yahboom@yahboom: ~  
[robot_state_publisher-3] [INFO] [1784805773.640503664] [robot_state_publisher]:  
  got segment imu_frame  
[robot_state_publisher-3] [INFO] [1784805773.640507845] [robot_state_publisher]:  
  got segment laser_frame  
[robot_state_publisher-3] [INFO] [1784805773.640512099] [robot_state_publisher]:  
  got segment lfwheel_Link  
[robot_state_publisher-3] [INFO] [1784805773.640516316] [robot_state_publisher]:  
  got segment rfwheel_Link  
[ekf_node-5] Failed to meet update rate! Took 0.050754432000000002101seconds. Tr  
y decreasing the rate, limiting sensor output frequency, or limiting the number  
of sensors.  
[joint_state_publisher-4] [INFO] [1784805779.911226012] [joint_state_publisher]:  
  Waiting for robot_description to be published on the robot_description topic...  
^[^A[camera_driver-2] [INFO] [1784805790.388324879] [esp32_ip_camera_publisher]:  
  camera node started, camera_url: http://192.168.12.37:81/stream  
[camera_driver-2] [WARN] [1784805793.381560579] [esp32_ip_camera_publisher]: Cam  
era not opened, trying reconnect... (next retry in 1.0s)  
[camera_driver-2] [INFO] [1784805795.017488392] [esp32_ip_camera_publisher]: IP  
camera stream opened successfully
```

Start the color calibration program:

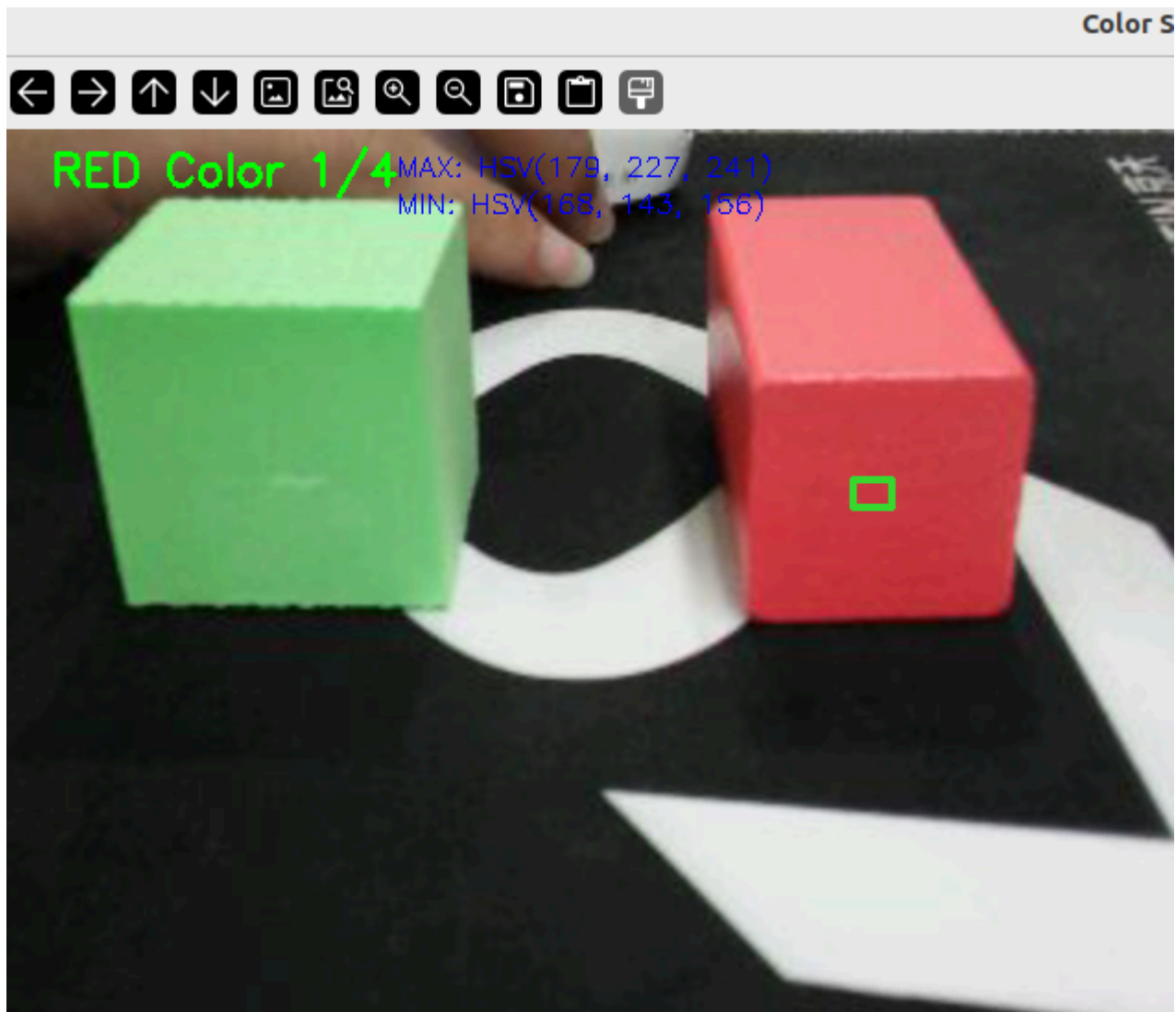
```
ros2 run multi_brains color_selector
```

After successful startup, the currently calibrating color will be displayed in the upper-left corner:

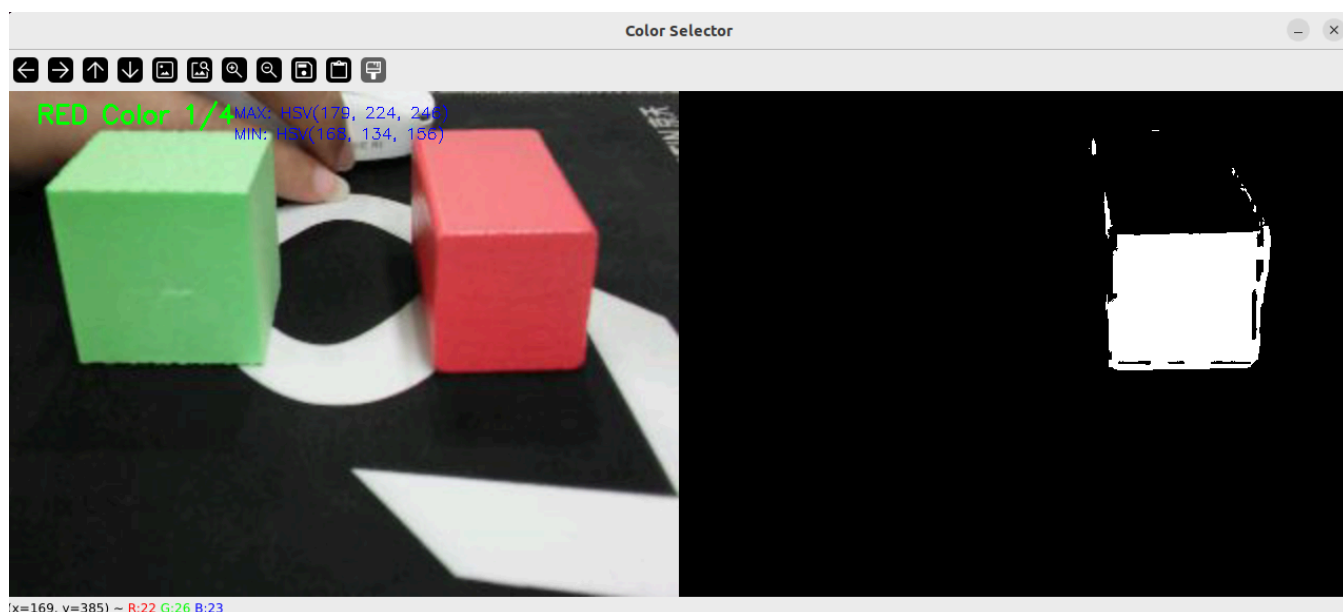
```
yahboom@yahboom:~$ ros2 run multi_brains color_selector
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFORM=wayland
to run on Wayland anyway.
[INFO] [1784806010.881335057] [color_selector]: Color Selector started | Topic:
/camera/image_raw
[INFO] [1784806010.881820495] [color_selector]: HSV file: /home/yahboom/workspac
es/MicroROS-V2-ROS2-SRC/microros_car_v2_ws/src/multi_brains/config/color_hsv.txt
[INFO] [1784806010.882159852] [color_selector]: Color Sampling Pan-Tilt initial-
ized: topic=/cmd_ptz_angle, s1=0.0, s2=-70.0
[INFO] [1784806010.882497223] [color_selector]: =====
=====
[INFO] [1784806010.882816699] [color_selector]: Usage Instructions:
[INFO] [1784806010.883127503] [color_selector]: 1. Select the target color area
[INFO] [1784806010.883498635] [color_selector]: 2. Release the mouse and view de
[INFO] [1784806010.883959597] [color_selector]: 3. Press [SPACE] to save the cur
[INFO] [1784806010.884564200] [color_selector]: 4. Press [E] to skip the current
[INFO] [1784806010.884976226] [color_selector]: 5. Press [Q] to exit the program
[INFO] [1784806010.885395847] [color_selector]: =====
=====
[INFO] [1784806011.693572282] [color_selector]: Color Sampling Pan-Tilt initiali
zation complete: s1=0.0, s2=-70.0
yahboom@yahboom:~$
```



Use the mouse to select a region (this region can only contain one color):



After releasing the mouse, the binarized image will be displayed immediately:



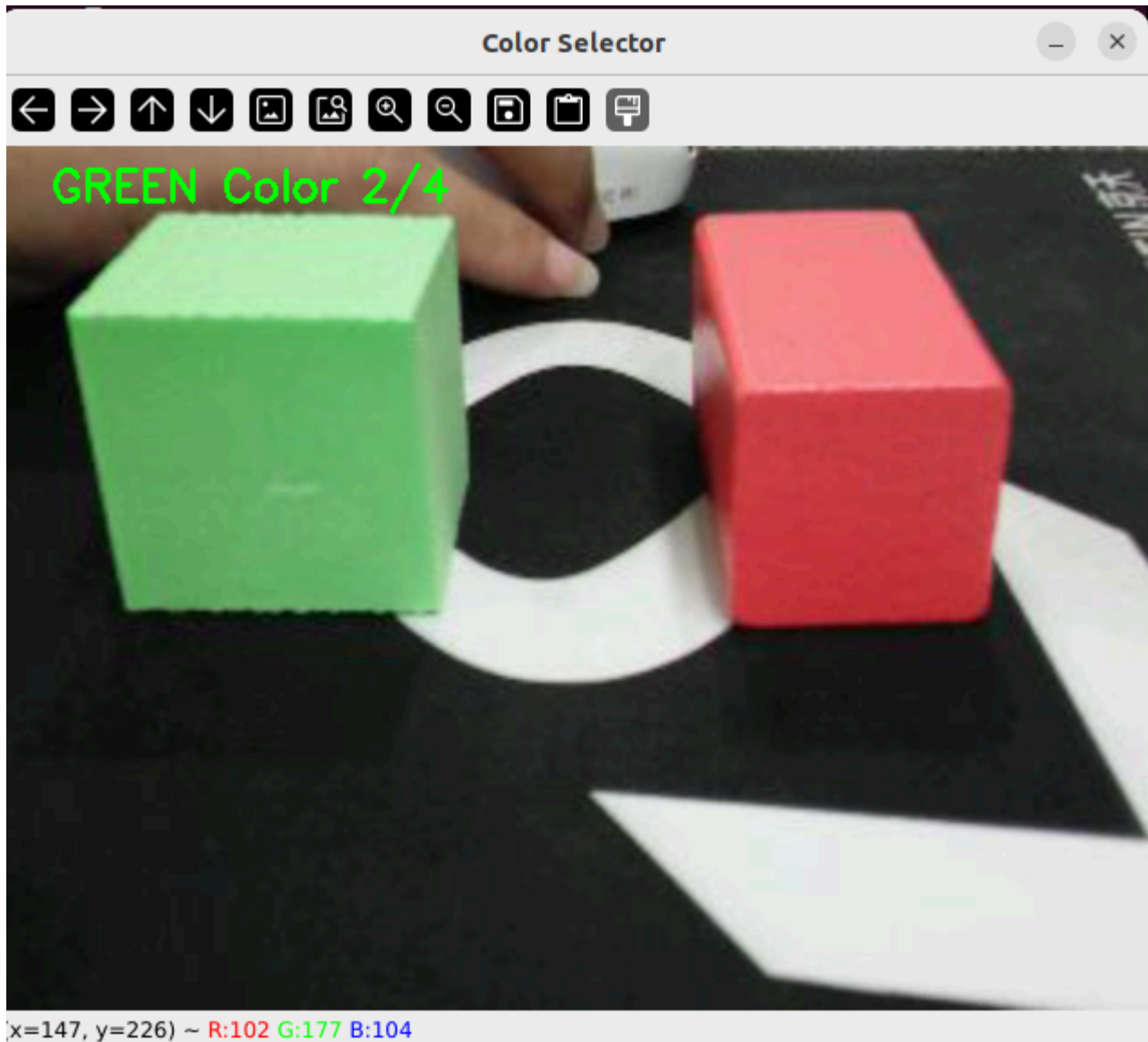
After confirming that the color is properly calibrated, press the `space` key, and the red HSV value will be written to the first line of the parameter file:

```
~/microros_ws/src/multi_brains/config/color_hsv.txt
```

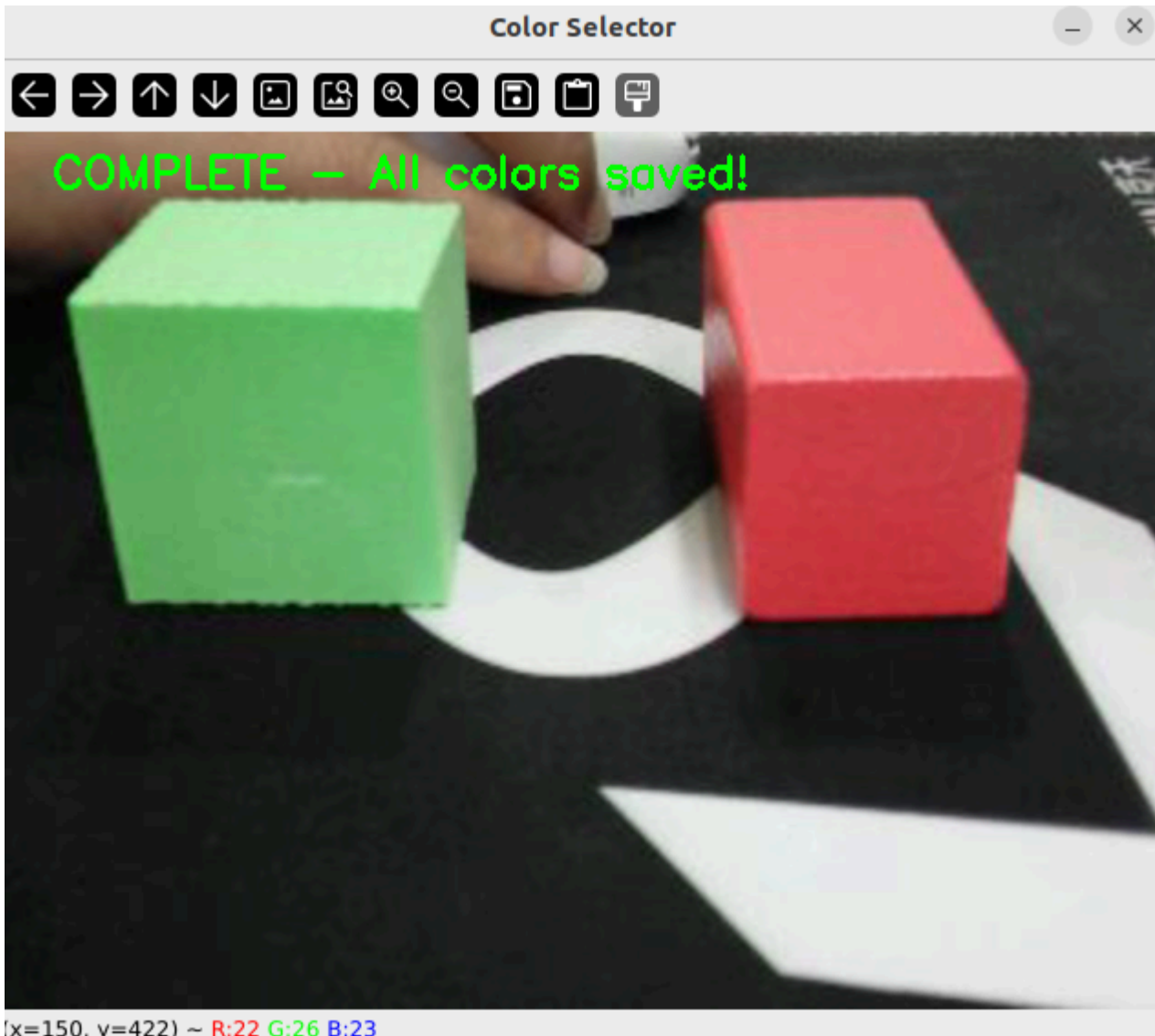
```
color_hsv.txt
~/workspaces/MicroROS-V2-ROS2-SRC/microros_car_v2_ws/...

1 0, 118, 99, 11, 205, 187
2 35, 43, 46, 77, 255, 255
3 100, 43, 46, 124, 255, 255
4 26, 43, 46, 34, 255, 255
```

Then move on to calibrating the next color. Continue calibrating the remaining colors in the same way, and the HSV values will be written to the second, third, and fourth lines of the parameter file:



When the last color calibration is completed, "all colors saved!" will be displayed in the upper-left corner:



Finally, you can press the `Q/q` key to exit the program.

4. Configure large model related API keys

All large model features require configuring the corresponding large model API keys. This chapter contains a lot of content and will not be elaborated here. Refer to the tutorial **【AI Large Model Development】 - 【2. Configure API-KEY】**.

5. Start the text interaction client

Prerequisite: configure the large model related API keys

Currently, the `multi_brains` package uses terminal text interaction instead of speech recognition and voice wake-up. The core node is responsible for connecting to Dify, parsing actions, and controlling the robot, while the terminal client is only responsible for inputting text and displaying the robot's replies.

Start the agent core node:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

Open another terminal and start the text interaction client:

```
ros2 run multi_brains terminal_client
```

After startup, you can directly input natural language commands, for example:

```
Follow the red line  
Look at what's ahead  
Move forward 2 meters then turn left 90 degrees
```